

ValeCare Backend Features Document: Django Implementation

Django REST Framework Implementation

Prepared for the SRHR Digital Solutions Project

Overview

This document translates the system requirements for ValeCare, drawn from the user journeys, into concrete backend features, organized as Django apps. Each app is scoped to one requirement group so backend fellows can divide the work without stepping on each other's models.

Every feature works for an anonymous guest. A lightweight, optional account is what lets a user's history, chat sessions, bookmarks, and cycle or pregnancy logs, follow her across visits and devices.

Technology Stack

- Django with Django REST Framework for the API layer, one app per feature group
- PostgreSQL for the database, with PostGIS left out for the MVP in favor of a simple haversine query
- Device token based session identification, extended with a username and passphrase for accounts that choose to register
- Django's built in admin for content moderators to manage ContentItem and EmergencyGuide records
- drf-spectacular or a similar tool for generating API documentation the frontend team can work from
- Standard Django test runner and factory_boy for model and endpoint tests

Design Principles

- Privacy by default: avoid storing a name, phone number, or email anywhere in the system.
- Guest by default, accounts by choice: every endpoint that does not touch saved history stays open to a guest device token. Only history bearing endpoints require a registered account.
- Open access: no field in any app filters content by age. All verified content is available to every user.
- Verified content only: is_verified and version fields gate what the triage assistant is allowed to draw from.
- Simple over clever: a haversine query instead of a geospatial extension, generic reminder text instead of a heavier notification system, for the MVP.

App Structure

Django App	Purpose	Key Models	Serves
accounts	Guest sessions and optional registered accounts	UserProfile	REQ-1.2, REQ-NF-1
education	SRHR learning content, bookmarks, and reading progress	ContentItem, ContentTag, Bookmark, ReadingProgress	REQ-1.1
triage	Symptom checker and AI assisted chat	SymptomEntry, TriageSession, TriageMessage	REQ-1.3, REQ-2.1 to 2.3
providers	Provider directory and search or filter	Provider, ProviderReview	REQ-3.1, REQ-3.3
appointments	Pseudonymous booking or inquiry	Appointment	REQ-3.2
support	Trusted adult or safe person connection	SafeContact, CounselorChat	REQ-3.4
emergency	Offline syncable emergency guidance content	EmergencyGuide	REQ-3.5, REQ-NF-2
followup	Follow up reminders and private closure	FollowUpTask	REQ-4.1 to 4.2

tracker	Menstrual cycle and pregnancy tracking	CycleEntry, PregnancyEntry	REQ-5.1 to 5.5, REQ-6.1 to 6.4
---------	--	----------------------------	--------------------------------

Feature 1: Guest Access and Registered Accounts (accounts app)

Serves REQ-1.2, REQ-NF-1.

A user can use the entire app anonymously, or create a lightweight account if she wants her history to follow her across visits and devices. A device generated token tracks the session either way. A registered account simply links that token to a username and passphrase instead of leaving it anonymous.

Model: UserProfile

Fields: device_token (CharField, unique, indexed), account_type (CharField, choices: guest, registered, default guest), username (CharField, unique, nullable), passphrase_hash (CharField, nullable), recovery_code_hash (CharField, nullable), preferred_language (CharField), created_at (DateTimeField).

API

- POST /api/accounts/session/, creates or restores a guest device token
- POST /api/accounts/signup/, creates a registered account from a username and passphrase, returns a one time recovery code
- POST /api/accounts/login/, verifies the username and passphrase and links the current device token to the account
- POST /api/accounts/recover/, verifies the username and recovery code, accepts a new passphrase, and issues a new recovery code
- PATCH /api/accounts/profile/, updates language, username, or passphrase
- POST /api/accounts/delete-request/, verifies the current passphrase and returns a short lived confirmation token
- DELETE /api/accounts/profile/?confirmation_token=, deletes the account and cascades the removal across triage sessions, bookmarks, and tracker entries tied to it

Notes

- No name, phone, or email field anywhere in this app. Account recovery runs on a one time code instead of a contact method.
- The recovery code is shown once, right after sign up or right after a recovery, and is stored hashed the same way the passphrase is, so the backend can verify it without ever being able to display it again.
- A guest device_token that never signs up has nothing tied to it beyond the current session. There is no server side record to recover if it is lost.
- Account deletion is a two step call by design, a request that confirms the passphrase and a follow up delete with the confirmation token, so a device left unlocked cannot delete an account in one accidental tap.

Feature 2: SRHR Learning Content (education app)

Serves REQ-1.1.

Model: ContentItem

Fields: title (CharField), body (TextField), category (CharField, choices: srhr_basics, myths_vs_facts, emergency_care, pregnancy, family_planning), language (CharField), is_verified (BooleanField, default False), reviewed_by (CharField, optional).

Model: ContentTag

Fields: name (CharField), content_items (ManyToManyField to ContentItem).

Model: Bookmark

Fields: user (ForeignKey to UserProfile), content_item (ForeignKey to ContentItem), created_at (DateTimeField).

Model: ReadingProgress

Fields: user (ForeignKey to UserProfile), content_item (ForeignKey to ContentItem), status (CharField, choices: not_started, in_progress, completed), updated_at (DateTimeField).

API

- GET /api/education/content/?category=&language=, no authentication required
- GET /api/education/content/{id}/
- POST /api/education/bookmarks/, GET /api/education/bookmarks/, DELETE /api/education/bookmarks/{id}/
- POST /api/education/progress/, GET /api/education/progress/

Content browsing needs no authentication at all, a guest reads exactly what a registered user reads. Bookmarks and reading progress require a registered account, since the entire point of both is remembering something across visits.

Feature 3: Symptom Checker and AI Assisted Triage (triage app)

Serves REQ-1.3, REQ-2.1, REQ-2.2, REQ-2.3.

Model: SymptomEntry

Fields: user (ForeignKey to UserProfile), plain_language_input (TextField), matched_symptom_tags (ManyToManyField), created_at (DateTimeField).

Model: TriageSession

Fields: user (ForeignKey), status (CharField, choices: open, resolved), created_at, updated_at.

Model: TriageMessage

Fields: session (ForeignKey to TriageSession), sender (CharField, choices: user, assistant), message (TextField), is_verified_source (BooleanField), created_at.

API

- POST /api/triage/sessions/, opens a new session
- POST /api/triage/sessions/{id}/messages/, posts a message and gets the assistant's reply
- GET /api/triage/sessions/{id}/messages/, returns message history
- DELETE /api/triage/sessions/{id}/, closes and clears a session
- POST /api/triage/symptoms/, records a SymptomEntry

The assistant's replies are constrained to content sourced from verified ContentItem records wherever possible, and each assistant message carries is_verified_source so the frontend can visibly label it, directly addressing user distrust of unverified AI answers.

Feature 4: Provider Directory and Filtering (providers app)

Serves REQ-3.1, REQ-3.3.

Model: Provider

Fields: name (CharField), gender (CharField, choices: female, male, prefer_not_to_say), latitude (FloatField), longitude (FloatField), is_faith_sensitive (BooleanField), languages_spoken (CharField).

Model: ProviderReview

Fields: provider (ForeignKey), rating (IntegerField, 1 to 5), respectful_care_flag (BooleanField), comment (TextField, optional), submitted_by (ForeignKey to UserProfile), created_at.

API

Endpoint: GET /api/providers/?gender=&max_distance_km=&faith_sensitive=&lat=&lng=

Endpoint: POST /api/providers/{id}/reviews/, submits a ProviderReview

Distance filtering can be computed with a simple haversine query for the MVP rather than a full geospatial database extension.

Feature 5: Pseudonymous Appointment Booking (appointments app)

Serves REQ-3.2.

Model: Appointment

Fields: user (ForeignKey to UserProfile), provider (ForeignKey to Provider), requested_time (DateTimeField), status (CharField, choices: requested, confirmed, cancelled), reference_code (CharField, auto generated).

API

- POST /api/appointments/, creates a booking request
- GET /api/appointments/, lists the current user's appointments
- PATCH /api/appointments/{id}/, updates status, for example to cancel

The reference_code, not the user's identity, is what is shown to the provider. No name field is required to create an Appointment.

Feature 6: Trusted Adult or Safe Person Connection (support app)

Serves REQ-3.4.

Model: SafeContact

Fields: name (CharField), role (CharField, for example school counselor), is_verified (BooleanField).

Model: CounselorChat

Fields: user (ForeignKey), safe_contact (ForeignKey to SafeContact), status (CharField, choices: open, closed), created_at.

Kept as a separate app from clinical triage since the tone and purpose, a safe adult to talk to, not a diagnosis, are different. This models one trusted contact per user rather than a directory of many counselors with specialties or ratings, keeping the feature about a steady, safe relationship instead of a marketplace.

Feature 7: Offline and Low Data Emergency Guidance (emergency app)

Serves REQ-3.5, REQ-NF-2.

Model: EmergencyGuide

Fields: title (CharField), condition_tag (CharField, for example fgm_complication), steps (TextField), severity_level (CharField, choices: seek_immediate_care, self_care_ok), version (IntegerField), last_updated (DateTimeField).

API

Endpoint: GET /api/emergency/guides/sync/?since_version=

This endpoint is designed for bulk sync rather than one record at a time fetches, so the frontend can cache the full emergency guide set locally on first connection and refresh only when the version changes. This is what makes the guidance available even when a user's connectivity drops.

Feature 8: Follow Up Reminders and Private Closure (followup app)

Serves REQ-4.1, REQ-4.2.

Model: FollowUpTask

Fields: user (ForeignKey), related_session (ForeignKey to TriageSession, optional), due_at (DateTimeField), notification_text (CharField, default generic, for example "You have a check-in"), status (CharField, choices: pending, resolved, dismissed).

API

Endpoint: GET /api/followup/tasks/, returns the current user's pending reminders

notification_text is intentionally generic by default so a shared or family device does not reveal the reason for the reminder.

Feature 9: Cycle and Pregnancy Tracking (tracker app)

Serves REQ-5.1 through REQ-5.5, REQ-6.1 through REQ-6.4.

Lets a registered user log her menstrual cycle and, separately, keep a pregnancy journal, and get predictions and reminders back. This is available only to registered accounts, since the value of the feature is a history that persists between visits. A guest can still use the tracker in the app for the length of her session, those entries simply stay on the device and are never sent here.

Model: CycleEntry

Fields: user (ForeignKey to UserProfile), start_date (DateField), end_date (DateField, optional), flow_level (CharField, choices: light, medium, heavy), symptoms (CharField, comma separated tags), mood (CharField, optional), notes (TextField, optional), created_at (DateTimeField).

Model: PregnancyEntry

Fields: user (ForeignKey to UserProfile), start_date (DateField), due_date (DateField, computed from start_date), week_number (IntegerField, computed), journal_note (TextField, optional), created_at (DateTimeField).

API

- POST /api/tracker/cycles/, GET /api/tracker/cycles/, DELETE /api/tracker/cycles/{id}/
- GET /api/tracker/cycles/predictions/, returns the next estimated period and fertile window based on logged history
- POST /api/tracker/pregnancy/, GET /api/tracker/pregnancy/

Cross-Cutting Backend Standards

- Privacy by default: avoid storing a name, phone number, or email anywhere. Prefer device_token, username, and reference_code identifiers throughout.
- Content verification: ContentItem and EmergencyGuide both carry is_verified or version fields so unverified content can be excluded from what the triage assistant is allowed to draw from.
- Open access: no app filters what a user can see by age.
- History requires an account: triage sessions, bookmarks, reading progress, and tracker entries are only meaningfully persisted for registered accounts. Guest device tokens are not expected to survive past the current session.
- All endpoints above are intended as Django REST Framework ViewSets with the corresponding serializers. Permissions default to AllowAny for content browsing and emergency guidance, and a device token or account check for everything that reads or writes a specific user's data.